

AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

Choose an item.



Choose an item.

Assignment Title:	System Design of Agricare.		
Assignment No:	1	Date of Submission:	Click here to enter a date.
Course Title:	Object Oriented Programming 2		
Course Code:	CSC 2210	Section:	D
Semester:	Summer	2025-26	Course Teacher: DR. MD IFTEKHARUL MOBIN

Declaration and Statement of Authorship:

1. I/we hold a copy of this Assignment/Case-Study, which can be produced if the original is lost/damaged.
2. This Assignment/Case-Study is my/our original work and no part of it has been copied from any other student's work or from any other source except where due acknowledgement is made.
3. No part of this Assignment/Case-Study has been written for me/us by any other person except where such collaboration has been authorized by the concerned teacher and is clearly acknowledged in the assignment.
4. I/we have not previously submitted or currently submitting this work for any other course/unit.
5. This work may be reproduced, communicated, compared and archived for the purpose of detecting plagiarism.
6. I/we give permission for a copy of my/our marked work to be retained by the faculty for review and comparison, including review by external examiners.
7. I/we understand that Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to expulsion from the University. Plagiarized material can be drawn from, and presented in, written, graphic and visual form, including electronic data, and oral presentations. Plagiarism occurs when the origin of their material used is not appropriately cited.
8. I/we also understand that enabling plagiarism is the act of assisting or allowing another person to plagiarize or to copy my/our work.

* Student(s) must complete all details except the faculty use part.

** Please submit all assignments to your course teacher or the office of the concerned teacher.

Group Name/No.:

No	Name	ID	Program	Signature
1	Lutfur Rahman	24-57054-1	BSc [CSE]	
2	Md. Jubair Hasan Tamim	23-51855-2	BSc [CSE]	
3	Asfi Sabrin Neha	24-56321-1	BSc [CSE]	
4	Md Julfiker ahmad Rafi	23-52116-2	BSc [CSE]	

Faculty use only

FACULTY COMMENTS	Marks Obtained	
	Total Marks	

CSC 2210 — Object Oriented Programming 2

Project Assignment 01: System Design & Report Submission

American International University–Bangladesh (AIUB)

Faculty of Science and Technology · Department of Computer Science

Semester: _____

1.What We Are Building (AgriCare System)

AgriCare System is a digital Animal Healthcare Management System designed to help farmers easily report animal health problems and receive veterinary services. The system provides a platform where Farmers, Doctors, and Managers can perform their respective activities in an organized way.

Farmers can register in the system, report animal health problems, select an available Doctor, and choose a suitable service such as Online Advice or Farm Visit. They can also make payments for the selected service, track their service requests, and provide ratings and reviews to Doctors after the service is completed.

Doctors can manage their assigned animal problems through the system. They can view problem details, provide the required treatment or advice, mark a service as completed, or transfer a problem to another available Doctor if they are unable to solve it. Each Doctor also has a profile containing information such as specialization, experience, service fees, rating, and total reviews.

The Manager is responsible for managing the overall operation of the system. The Manager can approve newly registered Doctors, set their service fees, manage user accounts, monitor animal problem reports, and manage financial records. Doctors remain in Pending status until they are approved by the Manager, while Farmers can become active immediately after registration.

Financial management is also an important part of the AgriCare system. The Manager sets the fees for services such as Online Advice and Farm Visit, and the selected service amount is recorded with each service request. The system keeps track of the payment status and amount for every Farmer service request.

The system also maintains Doctor Payment records. The Manager can record payments made to Doctors based on the number of animals or services handled and the total amount. This helps the Manager maintain proper financial records and monitor the payments related to veterinary services.

Overall, AgriCare integrates Animal Healthcare Management, User Management, Service Management, and Financial Management into one system. It reduces manual work, makes veterinary services more accessible to Farmers, helps Doctors manage their services efficiently, and allows the Manager to monitor both operational and financial activities.

1.1 The Three Mandatory User Roles

The AgriCare system is designed with three different user roles, each having specific responsibilities and features. These roles are **Manager, Doctor, and Farmer**. Each user has role-based access to ensure that users can only perform the activities related to their responsibilities.

User 1 — MANAGER

The Manager is responsible for managing the overall operation of the AgriCare system. The Manager supervises users, monitors animal health service activities, manages Doctors, and maintains financial records.

#	Feature	Description
1	Doctor Management	Approves newly registered Doctors and sets their Online Advice and Farm Visit service fees.
2	User Management	Manages Doctor and Farmer accounts, including viewing user information and blocking users when necessary.
3	Report Management	Monitors and manages animal health problem reports submitted by Farmers.
#	Feature	Description
4	Payment Management	Monitors Farmer service payments and maintains payment records for Doctors based on the services they provide.
5	Service Monitoring	Monitors service requests and the overall progress of animal healthcare services

User 2 — DOCTOR

The Doctor is responsible for providing veterinary services to Farmers and handling the animal health problems assigned to them.

#	Feature	Description
1	Problem Management	Views animal health problems assigned to the Doctor by Farmers.
2	Service Completion	Provides the required treatment or advice and marks the problem as completed after providing the service
3	Problem Transfer	Transfers an assigned problem to another available Doctor when the Doctor is unable to solve it
4	Doctor Profile	Maintains information such as specialization, experience, service fees, rating, and total reviews.
5	Service History	Can manage and track the animal health services handled through the system.

User 3 — FARMER

The Farmer is the primary service receiver in the AgriCare system. Farmers use the system to report animal health problems and obtain veterinary services from available Doctors.

#	Feature	Description
1	Animal Problem Reporting	Reports animal health problems by providing the animal name, animal type, problem description, and other required information.
2	Doctor Selection	Views available Doctors and selects a suitable Doctor based on information such as specialization, experience, and rating.
#	Feature	Description
3	Service Selection & Payment	Selects Online Advice or Farm Visit and makes the required service payment.
4	Request Tracking	Views submitted service requests and checks their payment and service status.

5	Doctor Rating	Provides a rating and review to the Doctor after a service has been successfully completed.
---	---------------	---

3. Deliverables — What Goes in the Repository

There are **seven** deliverables. All seven live inside your GitHub repository.

#	Deliverable	Format	Where in repo
D1	Case Study (written scenario)	Markdown text	README.md
D2	Functional Requirements + User Stories (all 3 roles)	Markdown text	README.md
D3	UI Navigation Diagram	PNG image	/docs/diagrams/ + shown in README.md
D4	SQL Schema Diagram (tables, columns, PK/FK, relationships)	PNG image	/docs/diagrams/ + shown in README.md
D5	SQL Script (CREATE TABLE + sample INSERT + feature queries)	.sql file	/database/schema.sql
D6	UI Mockups / Form Designs (images of every planned form)	PNG images	/docs/screenshots/ + shown in README.md
D7	Report file (AIUB cover page + all chapters)	PDF	/docs/Project_Report.pdf
Bonus	ER Diagram (<i>optional, +2 marks</i>)	PNG image	/docs/diagrams/

D1.Case Study

Managing animal healthcare on a farm involves more than simply raising livestock. Farmers need to identify animal health problems, find suitable veterinary Doctors, arrange appropriate services, make payments, and keep track of the progress of each service request. When these activities are handled manually through phone calls, paper records, or scattered information, important health problems may be overlooked, service information can become difficult to track, and payment records may not be properly maintained. **AgriCare** solves these problems

by providing a centralized digital platform where Farmers, Doctors, and Managers can manage animal healthcare services in an organized and efficient way.

The system is built around three main actors. The **Manager** is responsible for the overall management of the platform. The Manager approves newly registered Doctors, sets their service fees, manages user accounts, monitors animal problem reports, and manages financial records. The **Doctor** provides veterinary services to Farmers. Doctors can view assigned animal problems, complete the required services, or transfer a problem to another available Doctor when necessary. The **Farmer** is the main service receiver who can report animal health problems, select a suitable Doctor, choose a service such as Online Advice or Farm Visit, make the required payment, track service requests, and rate the Doctor after the service is completed.

Every person who logs into the system is represented by a single **User**, uniquely identified by a UserID and described by attributes such as name, Gmail, password, role, and account status. Doctor-specific information is maintained through the **DoctorProfile**, which stores details such as specialization, experience, Online Advice Fee, Farm Visit Fee, rating, and total reviews. A Farmer's healthcare request is represented by a **ServiceRequest**, which contains information about the Farmer, selected Doctor, animal, problem description, service type, service amount, payment status, request status, and request date. The reported animal problem is also maintained through the **AnimalProblems** entity, which stores the problem details and its assigned Doctor. After a service is completed, the Farmer can submit a **DoctorReview**, which records the rating given to the Doctor.

Financial management is also an important part of AgriCare. The system records the amount paid by the Farmer for each requested service along with its payment status. The service fees are determined by the Manager based on the Doctor's Online Advice and Farm Visit fees. In addition, **DoctorPayments** records payments made to Doctors based on the number of animals or services handled and the total payment amount. This allows the Manager to maintain organized financial records and monitor both service payments and Doctor payments.

These entities are connected to support the complete healthcare workflow. One User can have a DoctorProfile when the user is a Doctor. A Farmer can create multiple ServiceRequests, while a Doctor can receive multiple service requests. Each ServiceRequest is associated with an animal problem and can have a corresponding DoctorReview after completion. DoctorPayments are linked to Doctors and allow the Manager to maintain their payment history. Through these relationships, AgriCare provides a complete platform for **animal healthcare management, service management, user management, Doctor management, and financial management**, reducing manual work and making veterinary services more accessible, organized, and transparent.

D2.Functional Requirements & User Stories

Functional Requirement List

Manager

1. **Manage Doctor Accounts** – The Manager can view, approve, and block Doctor accounts.
2. **Manage Farmer Accounts** – The Manager can view and block Farmer accounts when necessary.
3. **Approve Doctors and Set Fees** – The Manager can approve pending Doctors and set their Online Advice and Farm Visit fees.
4. **Manage Animal Problem Reports** – The Manager can view and monitor animal health problem reports submitted by Farmers.
5. **Monitor Service Requests** – The Manager can monitor the status of veterinary service requests.
6. **Manage Payments** – The Manager can monitor Farmer payments and maintain Doctor payment records. **Doctor**
 1. **View Assigned Problems** – The Doctor can view animal health problems assigned to them.
 2. **Complete Services** – The Doctor can provide the required service and mark the assigned problem as completed.
 3. **Transfer Problems** – The Doctor can transfer an assigned problem to another available Doctor if they are unable to solve it.
 4. **View Service Information** – The Doctor can view the details of the Farmer, animal, and reported health problem related to an assigned service.

Farmer

1. **Register and Log In** – The Farmer can create an account and log into the system.
2. **Report Animal Problems** – The Farmer can submit animal health problems with the necessary details.
3. **Select a Doctor** – The Farmer can view available Doctors and select a suitable Doctor.
4. **Select Service and Make Payment** – The Farmer can choose Online Advice or Farm Visit and make the required payment.
5. **Track Service Requests** – The Farmer can view submitted requests and check their payment and service status.
6. **Rate and Review Doctor** – After a service is completed, the Farmer can provide a rating and review for the Doctor.
7. **Log Out** – The Farmer can securely log out from the system.

(b) User Stories

1. As a Manager, I can manage Doctor accounts, so that only authorized Doctors can provide services through the system.

Details: The Manager can view registered Doctors, approve pending Doctor accounts, set their Online Advice and Farm Visit fees, and block accounts when necessary. The Doctor's status is updated accordingly.

2. As a Manager, I can manage Farmer accounts, so that unauthorized users can be restricted from using the system.

Details: The Manager can view Farmer account information and block a Farmer when necessary. Active Farmers can continue using the available services.

3. As a Manager, I can approve a Doctor, so that verified Doctors can provide veterinary services.

Details: A newly registered Doctor initially has a Pending status. The Manager reviews the Doctor's account and sets the service fees. After approval, the Doctor's status changes to Active.

4. As a Manager, I can monitor animal problem reports, so that I can keep track of veterinary service activities.

Details: The Manager can view reported animal problems along with information such as Farmer, animal name, animal type, problem description, assigned Doctor, and current status.

5. As a Manager, I can monitor service requests, so that I can track the progress of veterinary services.

Details: The Manager can view service requests and their current status, including payment and service completion information.

6. As a Manager, I can manage financial records, so that service payments and Doctor payments are properly tracked.

Details: The system records the amount and payment status of each Farmer service request. The Manager can also maintain Doctor payment records based on the number of animals or services handled and the total amount paid.

7. As a Doctor, I can complete an assigned service, so that the Farmer can know that the problem has been handled.

Details: After providing the required treatment or advice, the Doctor can mark the assigned problem as Completed. The related service request is also updated as completed.

8. As a Doctor, I can complete an assigned service, so that the Farmer can know that the problem has been handled.

Details: After providing the required treatment or advice, the Doctor can mark the assigned problem as Completed. The related service request is also updated as completed.

9. As a Doctor, I can transfer an assigned problem, so that another Doctor can handle it when I cannot solve it.

Details: The Doctor can select another available and active Doctor and transfer the assigned problem to them. The problem is then assigned to the selected Doctor.

10. As a Doctor, I can maintain my professional information, so that Farmers can choose a suitable Doctor.

Details: The Doctor's profile contains information such as specialization, experience, Online Advice Fee, Farm Visit Fee, rating, and total reviews.

11. As a Farmer, I can register and log in, so that I can access veterinary services through the system.

Details: The registration form collects required user information such as UserID, Name, Gmail, Password, and Role. Farmer accounts become Active after registration and can then log in.

12. As a Farmer, I can report an animal health problem, so that I can receive veterinary assistance.

Details: The Farmer provides information such as animal name, animal type, problem description, service type, and farm address. A new service request and animal problem record are created.

13. As a Farmer, I can select a Doctor and service type, so that I can receive the veterinary service I need.

Details: The Farmer can view available Doctors and select a Doctor based on information such as specialization, experience, and rating. The Farmer can then choose Online Advice or Farm Visit.

14. As a Farmer, I can make a service payment, so that my selected veterinary service can be requested.

Details: The system calculates the service amount based on the selected Doctor and service type. The payment status is recorded along with the service request.

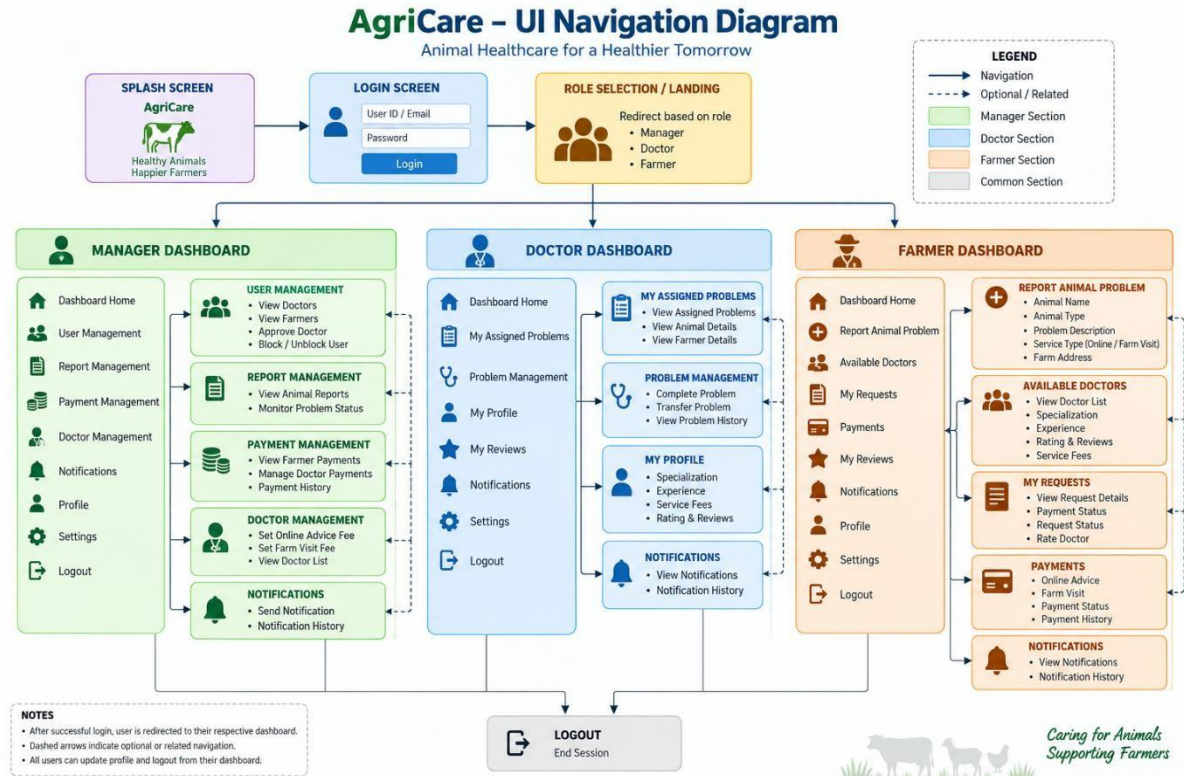
15. As a Farmer, I can track my service requests, so that I can know the current status of my reported problems.

Details: The Farmer can view submitted requests, including Request ID, animal information, Doctor ID, service type, amount, payment status, request status, and request date.

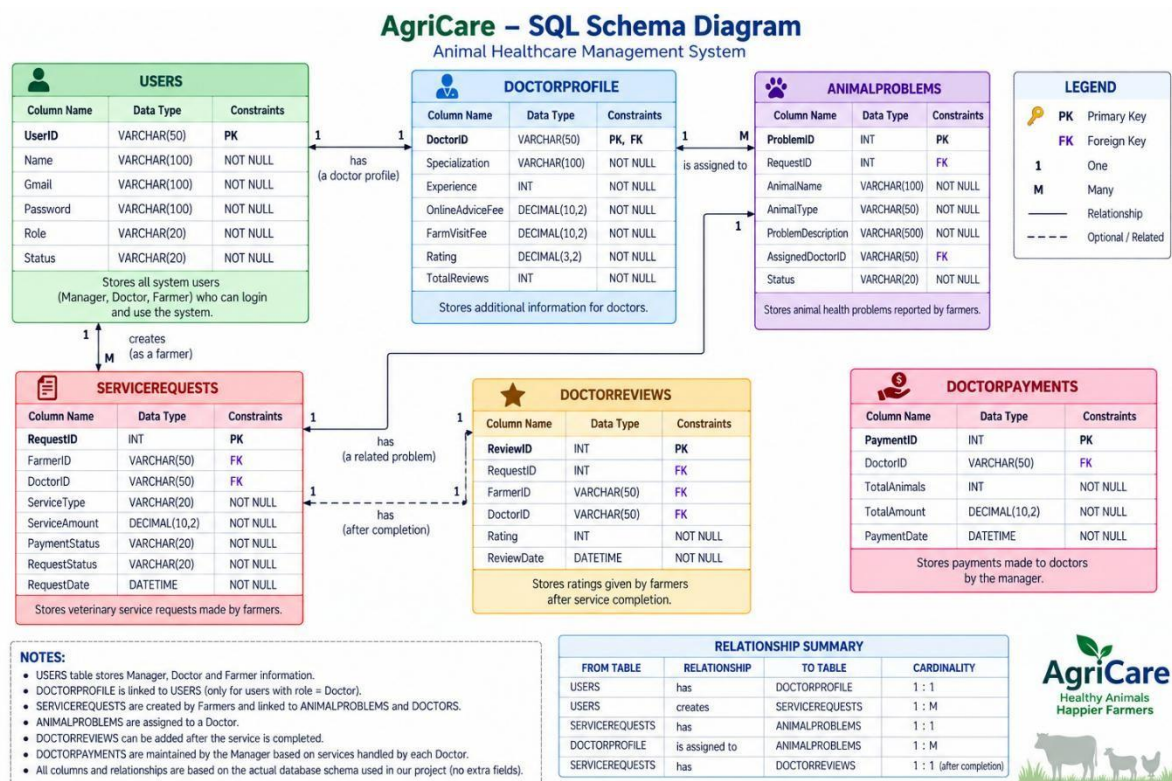
16. As a Farmer, I can rate and review a Doctor after a completed service, so that I can provide feedback about the service.

Details: The rating option becomes available after the related service request is **Completed**. The Farmer can give a rating from **1 to 5 stars**, and the Doctor's overall rating is updated based on the reviews received.

D3. UI Navigation Diagram



D4. SQL Schema Diagram



D5. SQL Script

1. USER

This entity stores information about all users of the AgriCare system.

Attributes:

- UserID (PK):** Unique identifier of the user.
- Name:** Name of the user.
- Gmail:** Gmail address of the user.
- Password:** User login password.
- Role:** Defines the user role (Manager, Doctor, or Farmer).
- Status:** Current account status of the user.

Relationship:

- A **Farmer** can create multiple **ServiceRequests**.
- A **Doctor** can have one **DoctorProfile**.
- A **Doctor** can receive multiple **DoctorReviews**.
- A **Doctor** can have multiple **DoctorPayments**.

Relationship Type:

- Users → ServiceRequests: **One-to-Many (1:M)**
- Users → DoctorProfile: **One-to-One (1:1)**
- Users → DoctorReviews: **One-to-Many (1:M)**
- Users → DoctorPayments: **One-to-Many (1:M)**

2. DOCTORPROFILE

This entity stores additional professional and service-related information about Doctors.

Attributes:

- **DoctorID (PK, FK):** Identifies the Doctor and references UserID in Users.
- **Specialization:** Doctor's specialization.
- **Experience:** Doctor's experience.
- **OnlineAdviceFee:** Fee for Online Advice service.
- **FarmVisitFee:** Fee for Farm Visit service.
- **Rating:** Overall rating of the Doctor.
- **TotalReviews:** Total number of reviews received by the Doctor. **Relationship:**
- A **DoctorProfile** belongs to one Doctor in the Users table.
- A **Doctor** can handle multiple **AnimalProblems**. **Relationship Type:**
- Users → DoctorProfile: **One-to-One (1:1)**
- Doctor → AnimalProblems: **One-to-Many (1:M)**

3. SERVICEREQUESTS

This entity stores the veterinary service requests created by Farmers.

Attributes:

- **RequestID (PK):** Unique identifier of the service request. • **FarmerID:** Identifies the Farmer who created the request.
- **DoctorID:** Identifies the selected Doctor.
- **AnimalName:** Name of the animal.
- **AnimalType:** Type of animal.
- **ProblemDescription:** Description of the animal health problem.
- **ServiceType:** Type of service, such as Online Advice or Farm Visit.
- **FarmAddress:** Address of the farm.
- **Amount:** Amount of the selected service.
- **PaymentStatus:** Current payment status.
- **RequestStatus:** Current status of the service request.
- **RequestDate:** Date and time when the request was created.

Relationship:

- A **Farmer** can create multiple ServiceRequests.
- A **Doctor** can receive multiple ServiceRequests.
- A ServiceRequest is related to one **AnimalProblem**.
- A ServiceRequest can have one **DoctorReview**. **Relationship Type:**
- Farmer → ServiceRequests: **One-to-Many (1:M)**
- Doctor → ServiceRequests: **One-to-Many (1:M)**
- ServiceRequests → AnimalProblems: **One-to-One (1:1)**
- ServiceRequests → DoctorReviews: **One-to-One (1:1)**

4. ANIMALPROBLEMS

This entity stores the animal health problems reported by Farmers.

Attributes:

- **ProblemId (PK):** Unique identifier of the animal problem.
- **FarmerId:** Identifies the Farmer who reported the problem.
- **AnimalName:** Name of the animal.
- **AnimalType:** Type of animal.
- **ProblemDescription:** Description of the animal health problem.
- **PaymentAmount:** Amount paid for the service.
- **PaymentStatus:** Payment status of the problem.
- **ProblemStatus:** Current status of the problem.
- **DoctorId:** Identifies the Doctor assigned to the problem.
- **ReportDate:** Date when the problem was reported.
- **ServiceType:** Type of veterinary service.
- **FarmAddress:** Address of the farm.
- **RequestID:** Identifies the related ServiceRequest. **Relationship:**
- A **Farmer** can report multiple AnimalProblems.
- A **Doctor** can be assigned multiple AnimalProblems.
- Each AnimalProblem is related to one ServiceRequest. **Relationship Type:**
- Farmer → AnimalProblems: **One-to-Many (1:M)**
- Doctor → AnimalProblems: **One-to-Many (1:M)**
- ServiceRequests → AnimalProblems: **One-to-One (1:1)**

5.DOCTORREVIEWS

This entity stores payment records made to Doctors.

Attributes:

- **DoctorID:** Identifies the Doctor receiving the payment.
- **TotalAnimals:** Number of animals/services handled by the Doctor.
- **TotalAmount:** Total amount paid to the Doctor.
- **PaymentDate:** Date when the payment was recorded. **Relationship:**
- A **Doctor** can have multiple DoctorPayment records. **Relationship Type:**
- Doctor → DoctorPayments: **One-to-Many (1:M)**

Normalization

USER Table:

1st Normal Form:

- UserID, Name, Gmail, Password, Role, Status are atomic.

2nd Normal Form:

- **PK:** UserID
- No partial dependency found as all non-key attributes fully depend on the primary key.

DOCTORPROFILE Table:

1st Normal Form:

- DoctorID, Specialization, Experience, OnlineAdviceFee, FarmVisitFee, Rating, TotalReviews are atomic. **2nd Normal Form:**

- **PK:** DoctorID

- No partial dependency found as all non-key attributes fully depend on the primary key.

SERVICEREQUESTS Table:

1st Normal Form:

- RequestID, FarmerID, DoctorID, AnimalName, AnimalType, ProblemDescription, ServiceType, FarmAddress, Amount, PaymentStatus, RequestStatus, RequestDate are atomic.

2nd Normal Form:

- **PK:** RequestID
- No partial dependency found as all non-key attributes fully depend on the primary key.

ANIMALPROBLEMS Table:

1st Normal Form:

- ProblemId, FarmerId, AnimalName, AnimalType, ProblemDescription, PaymentAmount, PaymentStatus, ProblemStatus, DoctorId, ReportDate, ServiceType, FarmAddress, RequestID are atomic.

2nd Normal Form:

- **PK:** ProblemId
- No partial dependency found as all non-key attributes fully depend on the primary key.

DOCTORREVIEWS Table:

1st Normal Form:

- ReviewID, RequestID, FarmerID, DoctorID, Rating, ReviewDate are atomic.

2nd Normal Form:

- **PK:** ReviewID
- No partial dependency found as all non-key attributes fully depend on the primary key.

DOCTORPAYMENTS Table:

1st Normal Form:

- DoctorID, TotalAnimals, TotalAmount, PaymentDate are atomic.

2nd Normal Form:

- **PK:** DoctorID
- No partial dependency found as all non-key attributes fully depend on the primary key.

D.6. UI Mockups / Form Designs

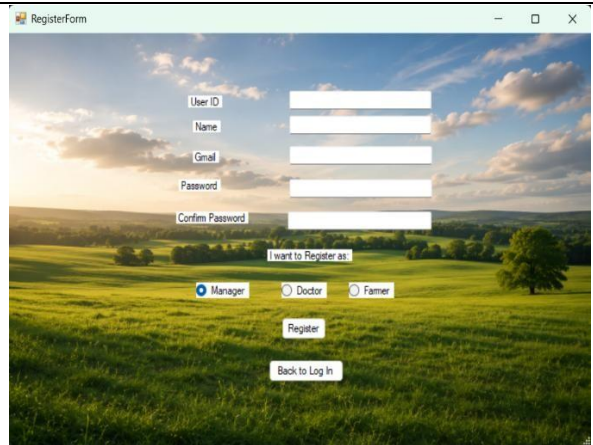


AgriCare

User ID

Password

User Log In



RegisterForm

User ID

Name

Gmail

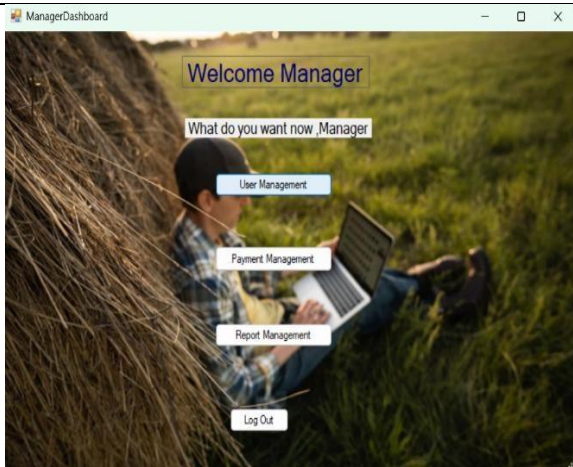
Password

Confirm Password

I want to Register as:

☒ Manager ☐ Doctor ☐ Farmer

User Sign In

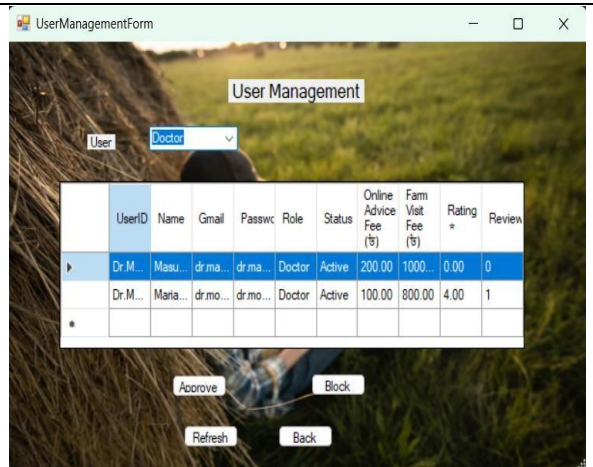


ManagerDashboard

Welcome Manager

What do you want now, Manager

Manager Dashedboard



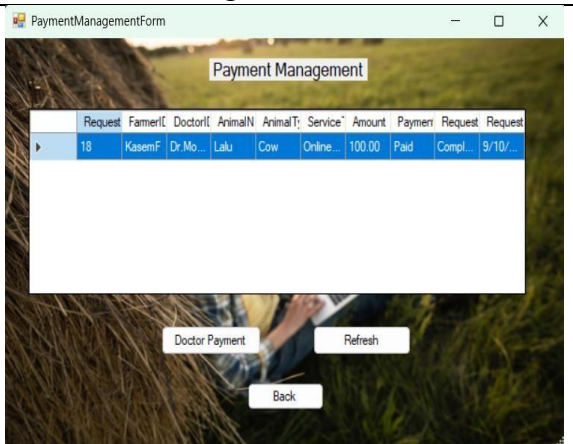
UserManagementForm

User Management

User

	UserID	Name	Gmail	Passw	Role	Status	Online Advice Fee (\$)	Farm Visit Fee (\$)	Rating *	Review
▶	Dr. M...	Masu...	dr.ma...	dr.ma...	Doctor	Active	200.00	1000...	0.00	0
	Dr. M...	Maria...	dr.mo...	dr.mo...	Doctor	Active	100.00	800.00	4.00	1

User Management

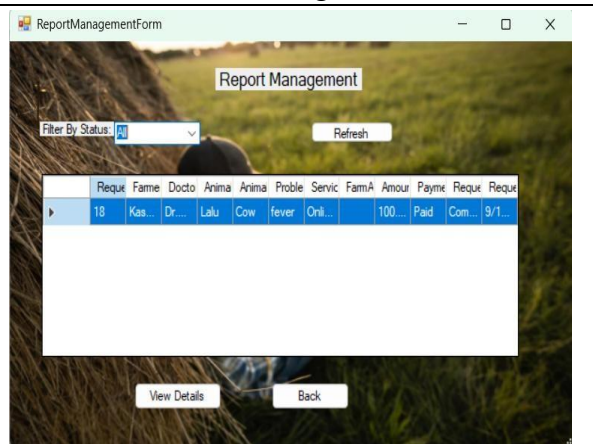


PaymentManagementForm

Payment Management

	Request	FarmerID	DoctorID	AnimalID	AnimalID	Service	Amount	Payment	Request	Request
▶	18	KasemF	Dr. Mo...	Lalu	Cow	Online...	100.00	Paid	Compl...	9/10/...

Payment Management



ReportManagementForm

Report Management

Filter By Status:

	Reque	Farme	Docto	Anima	Anima	Proble	Servic	FarmA	Amour	Payme	Reque	Reque
▶	18	Kas...	Dr...	Lalu	Cow	fever	Onli...	100...	Paid	Com...	9/1...	

Report Management

Welcome, Doctor

Report ID	Request ID	Farmer	Animal
*			

Assigned

Complete Report

Transfer

Log Out

Doctor Dashboard

Transfer problem

Select a Doctor

Masum Ahmed (Dr.Masum)

Done Cancel

Problem Transfer

Welcome Farmer

Report Animal Problem

My Requests

Available Doctors

Logout

Farmer Dashboard

Report Problem Form

Animal Name: Jesi

Animal Type: Goat

Problem Description: Can not stand

Service Type: Farm Visit

Farm Address: Nikunjo, Khlikhet

Select Doctor Cancel

Report Problem Form

Available Doctor

DoctorID	DoctorName	Specialization	Experience
▶ Dr.Mout	Maria Hossain M...	General Animal D...	0
Dr.Masum	Masum Ahmed	General Animal D...	0

Select Doctor Back

Doctor Selection Form

Payment

Doctor: Maria Hossain Mout

Service: Farm Visit

Amount: ₳800.00

Pay Now cancel

Payment

MyRequestsForm

My Service Requests

	Request ID	Animal	Animal Type	Problem	Doctor ID	Service Type	Amount	Payment	Status	Request Date
▶	19	Jesi	Goat	Can n...	Dr Mouri	Farm ...	800.00	Paid	Pending	9/11/...

Back

Pending request

MyRequestsForm

My Service Requests

	Request ID	Animal	Animal Type	Problem	Doctor ID	Service Type	Amount	Payment	Status	Request Date
▶	19	Jesi	Goat	Can n...	Dr Mouri	Farm ...	800.00	Paid	Compl...	9/11/...

Back

Rate Doctor *

After Complete Request

Rate Doctor

Doctor: Dr.Mouri

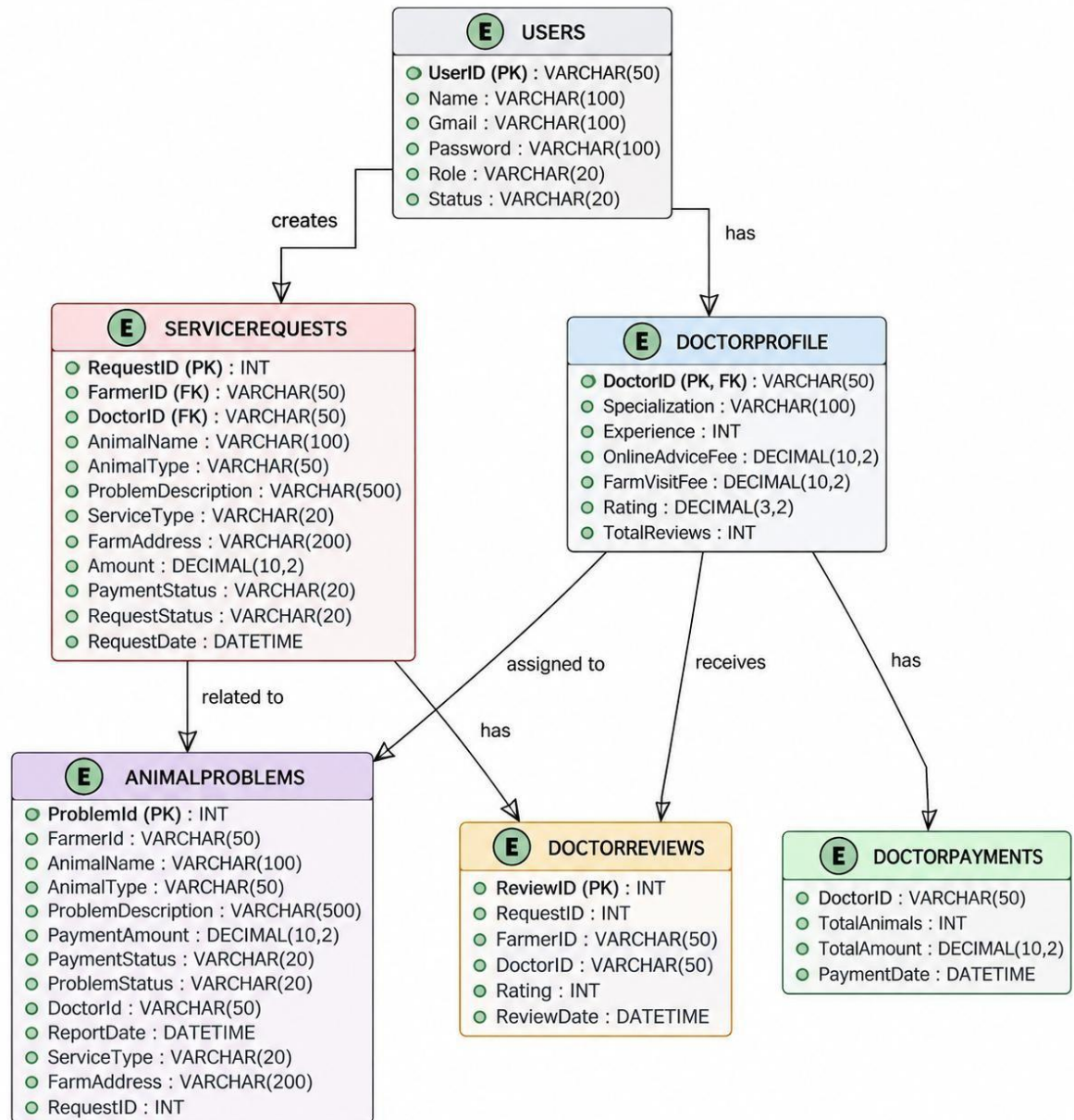
Animal: Jesi

Select Rating: ★★★★★ 5 Stars

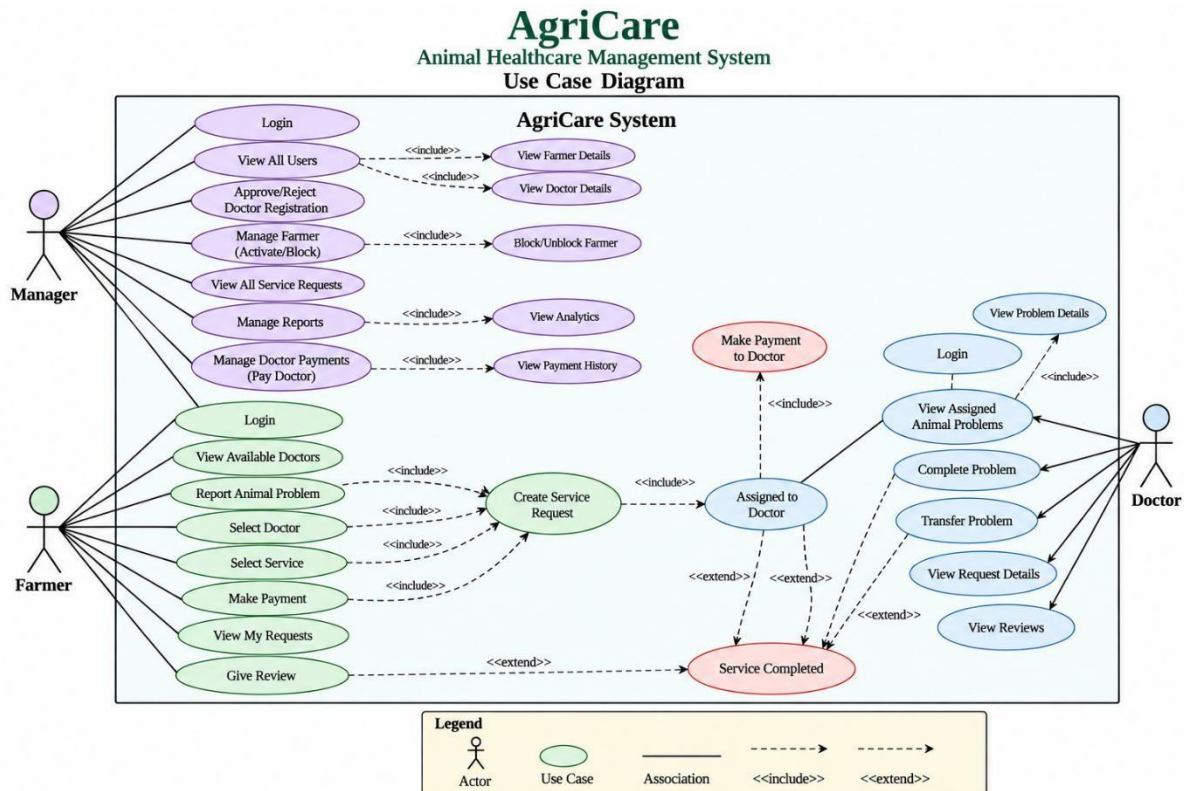
Submit Rating Cancel

Rating doctor

ER DIAGRAM



USE CASE DIAGRAM



The Use Case Diagram for the AgriCare Animal Healthcare Management System shows how the system is used by three key roles: Manager, Farmer, and Doctor. The Manager manages users, approves Doctor registrations, blocks or activates Farmers, manages animal problem reports, views service requests, and manages payments to Doctors. The Farmer can register and log in, report animal problems, view available Doctors, select a Doctor and service, make payments, view service requests, and give reviews after service completion. The Doctor can log in, view assigned animal problems, complete problems, transfer problems when necessary, and view reviews. The diagram also shows the flow from reporting an animal problem to selecting a Doctor, creating a service request, making payment, and handling the problem by the Doctor.

SQL Script (/database/schema.sql)

One runnable .sql file containing, in this order:

```
CREATE DATABASE AgriCareDB;
```

```
USE AgriCareDB;
```

1. USERS TABLE CREATE

```
TABLE Users
(
    UserID VARCHAR(50) PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Gmail VARCHAR(100) NOT NULL UNIQUE,
    Password VARCHAR(100) NOT NULL,
    Role VARCHAR(20) NOT NULL,
    Status VARCHAR(20) NOT NULL
);
```

2. DOCTORPROFILE TABLE

```
CREATE TABLE
DoctorProfile
(
    DoctorID VARCHAR(50) PRIMARY KEY,
    Specialization VARCHAR(100),
    Experience INT,
    OnlineAdviceFee DECIMAL(10,2),
    FarmVisitFee DECIMAL(10,2),
    Rating DECIMAL(3,2),
    TotalReviews INT,
    FOREIGN KEY (DoctorID)
REFERENCES Users(UserID)
);
```

3. SERVICEREQUESTS TABLE

```
CREATE TABLE ServiceRequests
(
    RequestID INT IDENTITY(1,1) PRIMARY KEY,
    FarmerID VARCHAR(50) NOT NULL,
    DoctorID VARCHAR(50) NOT NULL,
    AnimalName VARCHAR(100) NOT NULL,
    AnimalType VARCHAR(50) NOT NULL,
    ProblemDescription VARCHAR(500) NOT NULL,
    ServiceType VARCHAR(20) NOT NULL,
    FarmAddress VARCHAR(200),
    Amount DECIMAL(10,2) NOT NULL,
```

```

PaymentStatus VARCHAR(20) NOT NULL,
RequestStatus VARCHAR(20) NOT NULL,
RequestDate DATETIME NOT NULL,
FOREIGN KEY (FarmerID)
    REFERENCES Users(UserID),
FOREIGN KEY (DoctorID)
    REFERENCES Users(UserID)
);

```

4. ANIMALPROBLEMS TABLE CREATE

```

TABLE AnimalProblems
(
    ProblemId INT IDENTITY(1,1) PRIMARY KEY,
    FarmerId VARCHAR(50) NOT NULL,
    AnimalName VARCHAR(100) NOT NULL,
    AnimalType VARCHAR(50) NOT NULL,
    ProblemDescription VARCHAR(500) NOT NULL,
    PaymentAmount DECIMAL(10,2) NOT NULL,
    PaymentStatus VARCHAR(20) NOT NULL,
    ProblemStatus VARCHAR(20) NOT NULL,
    DoctorId VARCHAR(50) NOT NULL,
    ReportDate DATETIME NOT NULL,
    ServiceType VARCHAR(20) NOT NULL,
    FarmAddress VARCHAR(200),
    RequestID INT NULL,
    FOREIGN KEY (FarmerId)
        REFERENCES Users(UserID), FOREIGN
    KEY (DoctorId)
        REFERENCES Users(UserID),
    FOREIGN KEY (RequestID)
        REFERENCES ServiceRequests(RequestID)
);

```

5. DOCTORREVIEWS TABLE CREATE

```

TABLE DoctorReviews
(
    ReviewID INT IDENTITY(1,1) PRIMARY KEY,
    RequestID INT NOT NULL UNIQUE, FarmerID
    VARCHAR(50) NOT NULL,

```

```

    DoctorID VARCHAR(50) NOT NULL,
    Rating INT NOT NULL,
    ReviewDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (RequestID)
        REFERENCES ServiceRequests(RequestID),
    FOREIGN KEY (FarmerID)
        REFERENCES Users(UserID),
    FOREIGN KEY (DoctorID)
        REFERENCES Users(UserID)
);

```

6. DOCTORPAYMENTS TABLE CREATE

```

TABLE DoctorPayments
(
    DoctorID VARCHAR(50) NOT NULL,
    TotalAnimals INT NOT NULL,
    TotalAmount DECIMAL(10,2) NOT NULL,
    PaymentDate DATETIME NOT NULL,
    FOREIGN KEY (DoctorID)
        REFERENCES Users(UserID)
);

```

One SQL query per major feature, each with a `--` comment saying which feature it serves

Cover at minimum these query groups:

```

-- 1. LOGIN: verify credentials and get role for landing-page routing
-- 2. FILTER: products by price range
-- 3. FILTER: products by category + availability (stock > 0)
-- 4. SEARCH: products by keyword
-- 5. CART: add item / delete item / view cart with total

```

```

-- 7. SHOP EARNINGS: total sales grouped by shop (JOIN + GROUP BY + SUM)
-- 8. LOW STOCK ALERT: products where Stock < MinStock
-- 9. REVIEWS: all reviews for one product with reviewer name (JOIN)
-- 10. SUPER ADMIN: shops with average rating < 2.5 (JOIN + GROUP BY + HAVING)
-- 11. SUPER ADMIN: suspend/delete a shop owner
-- 12. OFFERS: active offers today with calculated discounted price

```

You must use **`JOIN`, `GROUP BY`, `HAVING`** and an **aggregate function** (SUM/AVG/COUNT) at least once each. Queries that are all `SELECT * FROM Table` score in the "Inadequate" band.

```
USE AgriCareDB;
```

```
-- 1. LOGIN: Verify credentials and get role and status
```

```
SELECT Name, Role, Status
FROM dbo.Users
WHERE UserID = @UserID
AND Password = @Password;
```

```
-- 2. AVAILABLE DOCTORS: View all active doctors
```

```
SELECT
    U.UserID AS [Doctor ID],
    U.Name AS [Doctor Name],
    ISNULL(DP.Specialization, 'General Animal Doctor') AS [Specialization],
    ISNULL(DP.Experience, 0) AS [Experience],
    ISNULL(DP.Rating, 0) AS [Rating],
    ISNULL(DP.TotalReviews, 0) AS [Total Reviews]
FROM dbo.Users U
LEFT JOIN dbo.DoctorProfile DP
    ON U.UserID = DP.DoctorID
WHERE U.Role = 'Doctor'
AND U.Status = 'Active'
ORDER BY U.Name;
```

-- 3. DOCTOR APPROVAL: Approve a pending doctor

```
UPDATE dbo.Users
SET Status = 'Active'
WHERE UserID = @UserID
AND Role = 'Doctor'
AND Status = 'Pending';
```

-- 4. REPORT MANAGEMENT: View animal problem reports

```
SELECT
    ProblemId AS [Report ID],
    FarmerId AS [Farmer ID],
    AnimalName AS [Animal],
    AnimalType AS [Animal Type],
    ProblemDescription AS [Problem],
    DoctorId AS [Doctor ID],
    ServiceType AS [Service Type], PaymentStatus
    AS [Payment],
    ProblemStatus AS [Status],
    ReportDate AS [Report Date] FROM
dbo.AnimalProblems
ORDER BY ReportDate DESC;
```

-- 5. MY REQUESTS: View service requests of a farmer

```
SELECT
    RequestID AS [Request ID],
    AnimalName AS [Animal],
    AnimalType AS [Animal Type],
    ProblemDescription AS [Problem],
    DoctorID AS [Doctor ID],
    ServiceType AS [Service Type],
    Amount AS [Amount],
    PaymentStatus AS [Payment],
    RequestStatus AS [Status],
    RequestDate AS [Request Date]
FROM dbo.ServiceRequests
WHERE FarmerID = @FarmerID
ORDER BY RequestDate DESC;
```

-- 6. SERVICE PAYMENT: Create a paid service request

```
INSERT INTO dbo.ServiceRequests
(
    FarmerID,
    DoctorID,
    AnimalName,
    AnimalType,
    ProblemDescription,
    ServiceType,
    FarmAddress,
    Amount,
    PaymentStatus,
```

```
RequestStatus,  
RequestDate  
)  
VALUES  
(  
    @FarmerID,  
    @DoctorID,  
    @AnimalName,  
    @AnimalType,  
    @ProblemDescription,  
    @ServiceType,  
    @FarmAddress,  
    @Amount,  
    'Paid',  
    'Pending',  
    GETDATE()  
);
```

-- 7. DOCTOR PAYMENT: Record payment made to a doctor

```
INSERT INTO dbo.DoctorPayments  
(  
    DoctorID,  
    TotalAnimals,  
    TotalAmount,  
    PaymentDate  
)  
VALUES
```

```
(  
    @DoctorID,  
    @TotalAnimals,  
    @TotalAmount,  
    GETDATE()  
);
```

-- 8. ASSIGNED PROBLEMS: View problems assigned to a doctor

```
SELECT  
    ProblemId AS [Report ID],  
    RequestID AS [Request ID],  
    FarmerId AS [Farmer ID],  
    AnimalName AS [Animal],  
    ProblemDescription AS [Problem]  
FROM dbo.AnimalProblems  
WHERE DoctorId = @DoctorId  
AND ProblemStatus = 'Assigned'  
ORDER BY ReportDate DESC;
```

-- 9. COMPLETE SERVICE: Mark an assigned animal problem as completed

```
UPDATE dbo.AnimalProblems  
SET ProblemStatus = 'Completed'  
WHERE ProblemId = @ProblemId  
AND DoctorId = @DoctorId  
AND ProblemStatus = 'Assigned';
```

```
UPDATE dbo.ServiceRequests
SET RequestStatus = 'Completed'
WHERE RequestID = @RequestID
AND DoctorID = @DoctorId;
```

-- 10. TRANSFER PROBLEM: Transfer an assigned problem to another active doctor

```
UPDATE dbo.AnimalProblems
SET DoctorId = @NewDoctorId
WHERE ProblemId = @ProblemId
AND DoctorId = @CurrentDoctorId
AND ProblemStatus = 'Assigned';
```

-- 11. DOCTOR REVIEW: Add farmer rating and update doctor rating

```
INSERT INTO dbo.DoctorReviews
(
    RequestID,
    FarmerID,
    DoctorID,
    Rating,
    ReviewDate
)
VALUES
(
    @RequestID,
    @FarmerID,
```

```
@DoctorID,  
@Rating,  
GETDATE()  
);  
  
UPDATE dbo.DoctorProfile  
SET  
    Rating =  
    (  
        SELECT CAST(  
            AVG(CAST(Rating AS DECIMAL(10,2)))  
            AS DECIMAL(10,2)  
        )  
        FROM dbo.DoctorReviews  
        WHERE DoctorID = @DoctorID  
    ),  
    TotalReviews =  
    (  
        SELECT COUNT(*)  
        FROM dbo.DoctorReviews  
        WHERE DoctorID = @DoctorID  
    )  
WHERE DoctorID = @DoctorID;
```